

Lecture 7: Distributions, Equilibrium, and AI Tools

Computational Bootcamp

John P. Ryan

Summer 2026

University of Wisconsin–Madison

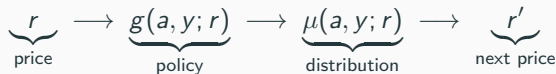
Today's goals

1. Compute the distribution a policy induces by simulation, and without any randomness at all
2. Close the model with prices
3. Develop good habits for using AI tools in research code - on today's model

Policy, Distribution, Price

The missing two-thirds

- We've computed $V(a, y)$ and $g(a, y)$. Now:
 - *What fraction of households are at the borrowing constraint?*
 - *Does a more generous unemployment benefit raise aggregate saving?*
 - *What is the interest rate?*
- The policy function answers what *one* household does at a state, not how many households
- A typical heterogeneous-agent model has **three** objects:



- General equilibrium is a fixed point of this Directed Acyclic Graph (DAG)
- Could also solve partial equilibrium (fixed, exogenous r) - no outer iteration

Today's economy: Huggett (1993)

- A continuum of households; income y alternates between employed ($y = 1$), unemployed ($y = b = 0.5$) on Markov chain Π ; one-period bond at rate r ; borrowing limit $\underline{a}$:

$$V(a, y) = \max_{a' \geq \underline{a}} \left\{ u(c) + \beta \sum_{y'} \Pi(y' | y) V(a', y') \right\}, \quad c = (1 + r)a + y - a'$$

- The household problem is the same as Aiyagari, with a possibly tighter borrowing limit
- The bond is in **zero net supply**: one household's borrowing is another's saving. Equilibrium is the r at which

$$A(r) \equiv \int a d\mu_r(a, y) = 0$$

- Solve the household problem, get μ , check whether $A(r) = 0$, adjust r .
- Policy is the hardest part, distribution is just accounting

The Distribution

Method 1: simulate a panel

- Put N households on the policy and run them forward
 1. Start each household somewhere (could be degenerate)
 2. Simulate pseudo random draws of y_{t+1} from $\Pi(y_{t+1} \mid y_t)$, keep fixed
 3. Use policy to simulate assets: $a_{t+1} = g(a_t, y_t)$
 4. Repeat T times; the cross-section at T is an estimate of μ
- **Ergodicity.** One household followed for a very long time visits states in the same proportions as many households at one date. So a long single path and a large cross-section estimate the same μ
- **Cost.** The estimate is a sample, so its error is $O(1/\sqrt{N})$: four times the households buys you two digits. Painful if the statistic you care about lives in a tail

Method 2: iterate the distribution directly (histogram)

- By LLN, there is no randomness in μ
- Put μ on a grid over (a, y) and let T^* be the transition matrix that $g(a, y)$ and Π induce: $T_{s,s'} = P(\text{state } s' \text{ tomorrow} \mid \text{state } s \text{ today})$. Then

$$\mu' = T^* \mu$$

- The stationary distribution is the fixed point - iterate it like the Bellman operator, from any μ_0 you like, until $\|\mu' - \mu\| < \text{tol}$
- Equivalently: μ is the eigenvector of T^* for eigenvalue 1
- No sampling noise, only discretization error, like quadrature vs Monte Carlo
- Cost is from the grid instead of sample. Typically much faster and more accurate than simulation.

- A state is a pair (a_i, y_j) . Two things happen, independently given today's state:
 - income moves to $y_{j'}$ with probability $\Pi_{jj'}$ (stochastic)
 - assets move to $a' = g(a_i, y_j)$ — deterministic, given the state
- So the entry is a product:

$$T_{(i,j) \rightarrow (k,j')}^* = \underbrace{\omega_{ik}}_{\text{asset part}} \times \underbrace{\Pi_{jj'}}_{\text{income part}}$$

- If $g(a_i, y_j)$ is exactly on grid point a_k , then $\omega_{ik} = 1$ and we are done (grid search)
- Any continuous choice of a' (an optimizer, an interpolated V , EGM) puts a' between two grid points or outside grid - have to split the mass between grid points
- **Note:** the grid for μ need not be the same as for V or g (often finer)
- Check: every row of T^* must sum to 1. If it doesn't, mass is leaking, distribution is wrong

The lottery weights (Young, 2010)

Say $a' = g(a_i, y_j)$ lands between grid points, $a_k \leq a' \leq a_{k+1}$. Split the household's mass.

- Put weight ω on a_k and $1 - \omega$ on a_{k+1} , choosing ω so the mass sits at a' on average:

$$\omega a_k + (1 - \omega) a_{k+1} = a' \quad \implies$$

$$\omega = \frac{a_{k+1} - a'}{a_{k+1} - a_k}$$

- The weight on the **lower** point is the relative distance to the **upper** one
- Check it works with $\omega + (1 - \omega) = 1$ and $\omega a_k + (1 - \omega) a_{k+1} = a'$
- Clamp a' to the grid on endpoints: if $a' < a_{min}$, $\omega = 1$ on a_{min} ; if $a' > a_{max}$, $\omega = 1$ on a_{max}

AI Tools and Best Practices

What these tools are for

- Three modes: **autocomplete** (Copilot), **chat** (Claude, ChatGPT, Gemini), **agents** that edit files under your supervision
- Some have free student tiers (e.g. the GitHub Student Developer Pack)
- Genuinely good at: boilerplate, plotting, Matlab → Julia translation, explaining errors, **writing tests**, rubber-ducking
- Best fit: fiddly-but-specified tasks — “split a value between two grid points” is exactly that shape
- The larger the task, more likely to fail and make mistakes.

Where AI tools fail (and it matters for us)

- **Hallucinated APIs**: functions or kwargs that don't exist, or belong to an old package
- **Subtly wrong math**: a misplaced expectation, a flipped lottery weight. Runs fine, looks plausible, is wrong
- **Confident nonsense**: models rarely say "I don't know"
- **Theory-naive code**: uses a simpler algorithm, doesn't apply theory correctly
- Often these failures are quiet.

Best practices for AI-assisted research code

1. **Commit before an agent touches your code**; review the diff after, roll back if it's wrong
2. **Never commit code you don't understand**. Ask the model to explain it, then verify
3. **Test a property**: a hand-computable case, comparative statics, a slow-but-correct version
4. **Ask for the test too**. Models write tests better than functions. You supply the property
5. **Work in small, verifiable chunks**: one function at a time
6. **Give good context**: the actual error, the signature, the package version
7. **State the deliverable**: describe specifically what you want an agent to produce
8. **State what to avoid**: deprecated syntax, packages outside your `Project.toml`, code the model shouldn't touch
9. **Iterate instead of one-shotting**: for anything nontrivial, ask for a plan or the equations first, then the code

- You are responsible for your results. “The AI wrote it” is not a defense
- Verify anything factual. Citations and theorem statements are common hallucination targets
- Disclosure policies are spreading — check them, and err toward disclosure
- Reproducibility matters *more*, not less, when code is machine-generated
- For this course: AI is allowed, but you should understand everything you submit

To the code!